AUDIT-SID: Artifact-Level Diagnostics for Semantic-ID Tokenizers in Generative Recommendation

Anonymous Author(s)

Abstract

Semantic-ID tokenizers are a core component of generative recommendation: items are mapped to discrete code sequences and retrieved by sequence generation. Yet most evaluations still emphasize final ranking metrics, which can hide codebook collapse, full-code collisions, collaborative-neighborhood mismatch, tail compression, or large prefix structures. We present AUDIT-SID, a mapping-first diagnostic toolkit for item-to-SID tokenizer artifacts. AUDIT-SID standardizes item mappings, metadata, interactions, and optional generator outputs, then reports five main artifact diagnostics: utilization, collision profile, semantic-collaborative alignment, head-tail capacity allocation, and structural cost proxies. In public-resource demonstrations, AUDIT-SID ingests a GRID/RQ-KMeans-style export and a bounded ReSID/GAOQ export, with controlled sanity tokenizers for interpretation. A same-item Musical case study on 23,742 items exposes sharply different artifact profiles: a GRID feature-text residual-k-means row has 3,749 unique full SIDs and a 0.9769 full-collision rate, while the bounded ReSID/GAOQ row is collision-free under its exported mapping. The contribution is a reusable artifact audit suite, not a new tokenizer or a full generative-recommender leaderboard.

CCS Concepts

• **Information systems** → **Recommender systems**; *Retrieval models and ranking*.

Keywords

semantic IDs, generative recommendation, tokenizer diagnostics, recommender systems

ACM Reference Format:

Anonymous Author(s). 2026. AUDIT-SID: Artifact-Level Diagnostics for Semantic-ID Tokenizers in Generative Recommendation. In . ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Semantic-ID (SID) tokenization has become a practical design point for generative recommendation. Systems such as TIGER map items into discrete code sequences so that a sequence model can generate candidate identifiers rather than score every catalog item directly [11]. This shift has made the tokenizer itself a first-order artifact. Recent work explores residual quantization frameworks, recommender-native encoding, differentiable SID learning, non-uniform quantization, qualified collisions, and continual tokenization [3–5, 7, 9, 14]. The broader identifier literature also shows that item indexing choices matter before generative retrieval is trained end-to-end [6], that semantic IDs can improve industrial ranking generalization [12], and that collaborative tokenization methods explicitly optimize behavioral alignment and assignment diversity [13, 15].

The evaluation practice has not caught up with this artifact boundary. Prior recommender tooling has argued for behavioral tests beyond NDCG-style aggregates [1], but SID tokenizers introduce a distinct artifact: the generated item address space. A tokenizer can support a usable downstream Recall@K while still hiding failure modes that are hard to diagnose from aggregate ranking metrics: underused codebooks, repeated full codes, semantically neat but behaviorally poor prefixes, tail items sharing too little capacity, or prefix structures that increase serving cost. These issues matter because SIDs are not merely features. They define the address space exposed to the generator.

This gap is especially awkward for resource reuse. A new tokenizer repository may expose checkpoints, item mappings, codebooks, or only intermediate feature files. Downstream users then need to answer basic engineering questions before training a generator: do all catalog items have valid codes, are many items aliased to the same full code, are collisions concentrated in the tail, and do prefix neighborhoods reflect user co-occurrence rather than only metadata taxonomy? Today these checks are usually reimplemented ad hoc inside each paper. The resulting evidence is difficult to compare because each method reports a different mixture of codebook usage, downstream ranking, and qualitative examples.

We present AUDIT-SID, a resource for auditing item-to-SID tokenizer artifacts before or alongside downstream recommendation evaluation. The goal is deliberately narrower than proposing a new tokenizer. AUDIT-SID asks a concrete question: given a public tokenizer artifact that maps items to code sequences, what can be said about its capacity use, collisions, behavioral alignment, head-tail allocation, and deployment cost without retraining a full generator?

The resource contribution is threefold. First, AUDIT-SID defines a small mapping-first interface for SID assignments, item metadata, interaction histories, and optional generator outputs. Second, it implements D1–D5a diagnostics for utilization, collisions, semantic-collaborative alignment, head-tail capacity, and structural serving-cost proxies, with D6 churn support for continual-tokenizer settings. Third, it provides public resource-demo evidence on GRID/RQ-KMeans-style, ReSID/GAOQ, and controlled sanity artifacts, including a same-item Musical case study. The paper follows the CIKM Resource four-page constraint, where appendices count toward the limit and detailed logs are therefore placed in the online artifact [2].

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Toolkit and Diagnostics

AUDIT-SID treats a tokenizer export as an artifact, not as an opaque component of a trained recommender. The minimum input is a table `sid_assignments(item_id, sid_0, ..., sid_L)`. Optional tables provide item metadata, interaction histories, and generator outputs. This contract lets adapters normalize heterogeneous repositories while keeping the audit independent of a particular training loop. Figure 1 shows how heterogeneous exports become comparable reviewer artifacts without retraining a downstream generator.

Artifact contract. The contract has three required checks and two optional extensions. The required checks are: every row must contain a stable item key, every SID level must be typed as a discrete token, and the item universe used by diagnostics must be joinable to the metadata and interaction slices. The optional metadata table supports semantic or category purity diagnostics. The optional interaction table supports collaborative-neighborhood diagnostics and head-tail splits. The optional `generator_outputs` table is deliberately separated from the mapping contract because many public tokenizer releases expose item codes without trained generator traces.

Validation before metrics. AUDIT-SID refuses to treat an adapter as evidence until it passes coverage and provenance checks: item-count agreement, missing-code counts, duplicate item keys, depth consistency, and declared caveats for feature sources or bounded training runs. This matters for the present resource paper because controlled exports, sanity tokenizers, and named-method mappings can all be loaded by the same metric runner but support different claims. The manifest therefore records whether a row is a runnable named method, a controlled stressor, an optional extension, or literature-only coverage context.

D1: utilization. AUDIT-SID reports per-level usage, prefix counts, entropy-style imbalance summaries, and dead or underused code indicators. These expose codebook collapse and uneven capacity even when full ranking metrics are unavailable.

D2: collision profile. AUDIT-SID measures duplicate full SIDs and prefix collisions at each depth. The reported full-collision rate is the fraction of items whose complete SID is shared by at least one other item. In the current resource version, D2 is a collision profile rather than a causal harm estimate. Interaction-qualified collision harm, as motivated by qualification-aware collision work [5], is reserved for the next artifact version. The current table still reports collisions because full code aliasing is an artifact-level fact: two items assigned the same complete address cannot be distinguished by a generator that emits only that address.

D3: semantic-collaborative alignment. Semantic grouping and collaborative usefulness need not agree. AUDIT-SID therefore computes a co-occurrence-based collaborative neighborhood proxy from training interactions and asks how often prefix neighborhoods recover each item's top-20 co-occurring neighbors. Metadata purity is retained only as an auxiliary diagnostic, not as the definition of collaborative quality.

D4: head-tail capacity. AUDIT-SID splits items into popularity buckets and measures whether full SID capacity and prefix structure are allocated differently across head, mid, and tail items. This helps separate genuine tokenizer behavior from simply restating the catalog popularity distribution.

D5a, D6, and D7. D5a reports structural cost proxies: SID length, level-wise prefix fan-out, duplicate codes, and trie-like expansion pressure. These are not real generator serving latencies. D6 computes churn between tokenizer refreshes and is useful for drift-aware continual-tokenization settings [3]. D7 is an interface hook for generator behavior, such as invalid paths, next-token entropy, and duplicate generated candidates; it requires `generator_outputs` and is not claimed by the current evidence.

Identifier foundations and recommender diagnostic frameworks motivate the resource framing, but they are not runnable tokenizer rows. Table 1 therefore focuses on SID artifact facets and marks which parts are demonstrated in v0.

3 Resource Demo and Case Study

The demo is designed as a diagnostic-resource check rather than a leaderboard. It asks whether AUDIT-SID can ingest real public SID artifacts from both a canonical residual-quantization line and a recent recommender-native line, then produce comparable audit tables with explicit caveats. GRID provides the open RQ-KMeans-style path [7]; ReSID provides the GAOQ line and processed Musical artifacts [9]. Sanity tokenizers are kept as controlled references for metric sensitivity.

Case-study protocol. The paper-facing case study uses the same 23,742 Musical items for both rows. The GRID row is a controlled feature-text export: ReSID processed feature text is embedded and passed through the official GRID-style residual k-means module. The ReSID row is a bounded FAMAE-to-GAOQ export on the same item universe. This protocol is intentionally modest. It gives reviewers a same-item diagnostic contrast while avoiding the stronger, unsupported claim that the paper reproduces every training choice of TIGER, GRID, or the full ReSID pipeline.

Table 2 compares two exports on the same 23,742 Musical items. The GRID row is intentionally labeled as feature-text: it uses processed feature text from the ReSID artifact path and the official MiniBatchKMeans-style residual quantization module, not a faithful raw-text TIGER reproduction. Under that controlled setup, the row exposes high collision pressure and limited tail capacity. The ReSID/GAOQ row exports one full code per item in this bounded run, yielding zero full-code collisions and higher D3-L1 collaborative prefix recovery. The D5a prefix column reports level-wise prefix counts, a structural proxy for trie expansion rather than real serving latency.

This contrast is useful precisely because the table is not a downstream ranking claim. It shows that the same audit interface can surface different artifact mechanisms: compact residual-k-means codes can collapse many items into shared full SIDs; recommender-native GAOQ can avoid full collisions in the exported mapping; and the tail-capacity column makes allocation pressure visible rather than inferred from item popularity. Additional artifact tables cover sanity controls, GRID scale checks, DACT churn, and a MovieLens

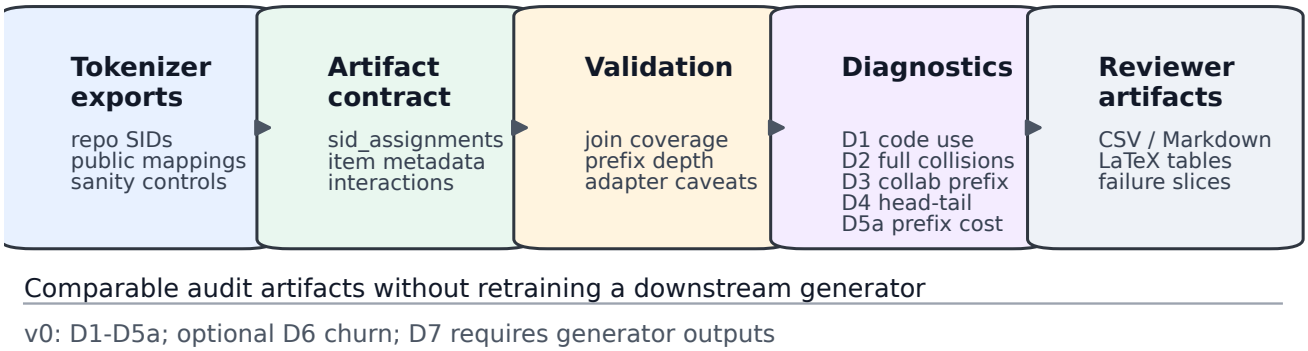

Figure 1: AUDIT-SID turns heterogeneous SID tokenizer exports into comparable audit artifacts. Exports are normalized into a mapping-first contract, validated for joinability and provenance, and summarized through D1–D5a diagnostics. D6 is optional churn evidence; D7 is enabled only when generator outputs or beam traces are provided.

Table 1: Paper-facing method/facet coverage. The table separates runnable v0 evidence from literature-motivated facets so that coverage context is not misread as reproduced tokenizer evidence.

Facet	Facet anchors, not all reproduced	v0 status	Diagnostics	Claim boundary
A. Canonical residual SID	TIGER, GRID, RQ-KMeans	Runnable controlled row	D1–D5a	Musical demo uses processed feature text; no raw-text TIGER reproduction.
B1. Collaborative / predictability aligned	ReSID, CoST, LETTER, DiscRec	Runnable bounded row	D1–D5a, D3	Bounded Musical export; no claim that all B1 systems are reproduced.
B2. Collision / utilization / capacity	QuaSID, AdaSID, CARD, CapsID	Backlog context	D1, D2, D4	Coverage only; no faithful named-method result is reported for these rows.
B3. Ranking / retrieval aligned	DIGER, joint search-rec SID	Interface hook	D3, D5a, D7	D7 needs generated candidates or beam traces, absent from v0.
B4. Bottleneck / interface variants	AsymRec, CapsID, Long SID	Future target	D4, D5a, D7	v0 covers structural D5a only, not generator behavior.
C. Drift / staleness	DACT, SID staleness	Optional smoke	D6	Continual tokenization evidence, not a B-cluster replacement.
D. Deployment / search surface	Snapchat SID, unified search-rec SID	Motivation only	D5a, D7	No production serving latency or online-impact claim.
R0/control layer	RecList, Elliot, How-to-Index, sanity SIDs, MovieLens smoke	Controls	D1–D5a	R0 papers are not tokenizer methods; sanity rows test metric behavior only.

Table 2: Same-item Musical artifact profiles on 23,742 items. Higher unique full codes, D3 L1 co-occurrence prefix recall, and D4 tail unique-SID ratio are better; lower D2 full-collision rate is better. D5a prefixes are level-wise prefix counts, not latency. This is not a downstream leaderboard: GRID feature-text uses ReSID processed feature text, and ReSID is a bounded GAOQ export.

System	Unique	D2 full	D3 L1	D4 tail	D5a prefixes
GRID feature-text	3,749	0.9769	0.0552	0.3695	64/3440/3749
ReSID bounded	23,742	0.0000	0.1535	1.0000	32/1280/23742

schema smoke, but those are placed in the repository rather than the four-page PDF.

What the extra artifact tables add. The repository tables are not backup evidence for hidden claims; they serve specific resource checks that do not need space in the PDF. Sanity controls show that D1–D5a react to intentionally extreme tokenizers rather than collapsing to constant values. GRID scale checks show that the adapter and metric runner handle larger item counts than the Musical case study. The DACT table demonstrates that the optional D6 churn interface can compare two SID refreshes. The MovieLens smoke test verifies that the schema is not tied to Amazon metadata. These tables support the artifact's usability claim, whereas Table 2 remains the only method-facing diagnostic case study in the main paper.

4 Availability and Limitations

The artifact repository contains the AUDIT-SID Python package, adapter scripts, metric runners, generated CSV, Markdown, and LaTeX tables, provenance logs, the BibTeX/source audit used for this draft, an MIT license, and a reviewer quickstart. The reviewer entry point is the tag `audit-sid-cikm-resource-v0.1`, which exposes a clean-checkout verifier for the published tables and figure. Large local artifacts are kept out of git and described through manifests so that reviewers can distinguish public reproducibility assets from local caches.

Table 3: Reviewer-facing artifact package. The table maps repository assets to verification actions and claims rather than asking reviewers to reproduce all upstream tokenizer training from scratch.

Reviewer action	Checked claim
Read quickstart and license	MIT-licensed resource with local verification commands.
Run tests and public verifier	Unit-tested D1–D5a/D6 code paths and published paper tables are checkable from a clean checkout.
Compare generated CSVs with Tables 1 and 2	Main numeric claims are traceable to generated files.
Inspect manifest and claim audit	Rows are labeled as runnable, controlled, optional, or literature-only evidence.
Open sanity, scale, churn, and MovieLens CSVs	Metric sensitivity, larger-item handling, D6 churn, and non-Amazon portability are documented.

The current evidence supports a conservative resource-demo claim. It does not support several stronger claims. AUDIT-SID does not reproduce TIGER end-to-end; the GRID Musical row is a controlled feature-text export. The ReSID Sports balanced GAOQ path was stopped because constrained k-means was CPU-bound under the available window, so the main ReSID evidence is the bounded Musical export. CARD is repaired only as a fallback/stressor path and should not be presented as faithful CARD reproduction [14]. D2 is a collision profile, not strict causal harm. D3 is a diagnostic proxy and is not yet proven monotonic with Recall@K or NDCG. D5a is a structural cost proxy without real generator-output latency, and D6 remains optional drift evidence.

These boundaries are intentional. The paper contributes an audit surface for tokenizer artifacts, not a new SID tokenizer, a complete generative-recommender evaluation, or an industrial online-impact study. Future versions should add interaction-qualified collision harm, generator output diagnostics, stronger same-dataset method coverage, and unified search and recommendation artifact checks [10]. Industrial SID deployment studies further motivate tracking these artifact properties as first-class engineering objects [8].

The most important limitation is interpretive rather than computational. Artifact diagnostics can reveal pressure points before expensive generator training, but they do not replace ranking evaluation. A tokenizer with cleaner capacity allocation can still be hard for a sequence model to predict, and a collision-heavy tokenizer can sometimes be rescued downstream. AUDIT-SID is therefore a preflight and debugging layer: it shows what kind of artifact a method produced, which slices deserve downstream evaluation, and which claims are unsafe to make from Recall@K alone.

Reviewer workflow. Reviewers first inspect the public manifest to distinguish named-method, controlled, optional, and literature-only rows. They then run the clean-checkout verifier and compare the generated figure plus published CSVs with the paper claims. Full metric rebuilds are optional and require local experiment caches.

Claim discipline. The resource separates three levels of statements. A *mapping statement* is directly supported by the SID-assignment table: for example, the number of unique full codes or the full-collision rate in Table 2. A *diagnostic statement* combines the mapping with metadata or interaction slices: for example, D3 collaborative prefix recovery or D4 tail allocation. A *system-quality statement* requires downstream generator training, ranking evaluation, or serving traces. The current paper makes the first two kinds of statements and explicitly does not make the third. This separation is the main design rule that keeps the resource useful even when upstream tokenizer releases differ in how much code, checkpoint state, or generator output they expose.

Why this is still resource-worthy. A resource paper does not need to prove that one tokenizer is better than another. It needs to make a reusable artifact easier to inspect, compare, and extend. AUDIT-SID contributes that layer for SID tokenizers through adapters, validators, diagnostics, generated tables, and a release checklist for mappings, metadata, interactions, optional generator outputs, and provenance.

Maintenance plan. The public artifact is organized so that new tokenizer adapters can be added without changing the diagnostic definitions. Each adapter emits the same normalized tables, declares its evidence role, and records feature or checkpoint provenance. Versioned manifests and fixed-name "latest" reports preserve both stable entry points and timestamped provenance for reviewers.

References

[1] Patrick John Chia, Maurizio Ferrari Dacrema, Maria Soledad Pera, and Jacopo Tagliabue. 2022. Beyond NDCG: Behavioral Testing of Recommender Systems with RecList. In *Companion Proceedings of the ACM Web Conference 2022*. arXiv:2111.09963 [cs.IR] doi:10.1145/3487553.3524215 arXiv:2111.09963.

[2] CIKM 2026 Organizing Committee. 2026. CIKM 2026 Resource Papers. https://cikm2026.diag.uniroma1.it/resource-papers/. Accessed 2026-05-19.

[3] Yuebo Feng, Jiahao Liu, Mingzhe Han, Dongsheng Li, Hansu Gu, Peng Zhang, Tun Lu, and Ning Gu. 2026. Drift-Aware Continual Tokenization for Generative Recommendation. *arXiv preprint arXiv:2603.29705* (2026). arXiv:2603.29705 [cs.IR] doi:10.48550/arXiv.2603.29705

[4] Junchen Fu, Xuri Ge, Alexandros Karatzoglou, Ioannis Arapakis, Suzan Verberne, Joemon M. Jose, and Zhaochun Ren. 2026. Differentiable Semantic ID for Generative Recommendation. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. arXiv:2601.19711 [cs.IR] doi:10.48550/arXiv.2601.19711 Accepted by SIGIR 2026; arXiv:2601.19711.

[5] Zheng Hu, Yuxin Chen, Yongsen Pan, Xu Yuan, Yuting Yin, Daoyuan Wang, Boyang Xia, Zefei Luo, Hongyang Wang, Songhao Ni, Dongxu Liang, Jun Wang, Shimin Cai, Tao Zhou, Fuji Ren, and Wenwu Ou. 2026. Stop Treating Collisions Equally: Qualification-Aware Semantic ID Learning for Recommendation at Industrial Scale. *arXiv preprint arXiv:2603.00632* (2026). arXiv:2603.00632 [cs.IR] doi:10.48550/arXiv.2603.00632

[6] Wenyue Hua, Yingqiang Ge, Shuyuan Xu, Zelong Ji, Jiayue Chen, and Yongfeng Zhang. 2023. How to Index Item IDs for Recommendation Foundation Models. *arXiv preprint arXiv:2305.06569* (2023). arXiv:2305.06569 [cs.IR] doi:10.48550/arXiv.2305.06569

[7] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvesh Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. *arXiv preprint arXiv:2507.22224* (2025). arXiv:2507.22224 [cs.IR] doi:10.48550/arXiv.2507.22224

[8] Clark Mingxuan Ju, Andy Lin, Adedamola Oyewole, Zeel Patel, Hyunjin Park, Jerome Arje, Zhe Dong, Anuj Bhardwaj, Rahul Rastogi, and Neil Shah. 2026. Semantic IDs for Recommendation Systems. *arXiv preprint arXiv:2604.03949* (2026). arXiv:2604.03949 [cs.IR] doi:10.48550/arXiv.2604.03949

[9] Yu Liang, Zhongjin Zhang, Yuxuan Zhu, Kerui Zhang, Zhiluohan Guo, Wenhang Zhou, Zonqi Yang, Kangle Wu, Yabo Ni, Anxiang Zeng, Cong Fu, Jianxin Wang, and Jiazhi Xia. 2026. Rethinking Generative Recommender Tokenizer: Recsys-Native Encoding and Semantic Quantization Beyond LLMs. *arXiv preprint arXiv:2602.02338* (2026). arXiv:2602.02338 [cs.IR] doi:10.48550/arXiv.2602.02338

[10] Gustavo Penha, Edoardo D'Amico, Marco De Nadai, Enrico Palumbo, Alexandre Tamborrino, Ali Vardasbi, Max Lefarov, Shawn Lin, Timothy Heath, Francesco Fabbri, and Hugues Bouchard. 2025. Semantic IDs for Joint Generative Search and Recommendation. In *Proceedings of the 19th ACM Conference on Recommender Systems*. arXiv:2508.10478 [cs.IR] doi:10.48550/arXiv.2508.10478 Late-Breaking Results track; arXiv:2508.10478.

[11] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems*. arXiv:2305.05065 [cs.IR] doi:10.48550/arXiv.2305.05065

[12] Adit Krishan Singh, Vamsi Meduri, Pranjal Awasthi, Yuanyuan Feng, Amee Trivedi, Afshin Rostamizadeh, and Sanjiv Kumar. 2023. Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations. *arXiv preprint arXiv:2306.08121* (2023). arXiv:2306.08121 [cs.IR] doi:10.48550/arXiv.2306.08121

[13] Bowen Wang, Shizhe Diao, Yan Feng, Baijun He, Chen Li, Xiyuan Wang, Jingang Fu, Haiyang Lyu, and Yonggang Chen. 2024. LETTER: A LLM-Enhanced Two-Tower Recommender System. *arXiv preprint arXiv:2405.07314* (2024). arXiv:2405.07314 [cs.IR] doi:10.48550/arXiv.2405.07314

[14] Yibiao Wei, Jie Zou, Pengfei Zhang, Xiao Ao, Weikang Guo, Zeyu Ma, and Yang Yang. 2026. CARD: Non-Uniform Quantization of Visual Semantic Unit for Generative Recommendation. *arXiv preprint arXiv:2604.26427* (2026). arXiv:2604.26427 [cs.IR] doi:10.48550/arXiv.2604.26427

[15] Han Zhu, Hao Liu, Zhiwei Liu, Wenqi Sun, Yingxue Zhang, Xinyang Fu, Weijie Liu, and Hui Xiong. 2024. Collaborative Semantic Tokenization for Generative Retrieval. *arXiv preprint arXiv:2404.14774* (2024). arXiv:2404.14774 [cs.IR] doi:10.48550/arXiv.2404.14774

GenAI Usage Disclosure

The authors used generative AI tools for drafting assistance, code-review support, and experiment-log organization. The reported claims, tables, and citations were checked against the artifact repository and cited sources during paper preparation.